Tuples (Exercises)

1.
```
t = (1, 2, 3)
```
```
t
```
(1, 2, 3)

2.
```
()
```
_____

3.
```
type(())
```
_____

4.
```
x, y, z = t
```
```
x
```
_____

5.
```
y
```
_____

6.
```
u = x, y
```
```
u
```
_____

7.
```
a = 1
b = 2
a, b = b, a
a
```
_____

8.
```
b
```
_____

9.
```
(2 + 2 + 2)
```
_____

10.
```
(2 + 2)
```
_____

11.
```
(2)
```
_____

12.
```
(2, 2, 2)
```
_____

13.
```
(2, 2)
```
_____

14.
```
(2,)
```
_____

Tuples

A *tuple* is almost exactly the same as a list, but it is delimited by round parentheses `(` and `)`. The difference is that a tuple, unlike a list, is *immutable*: it cannot be modified. This is helpful for a couple of reasons:

- The same tuple can have several names without any risk of the aliasing problems described previously under Equality.
- Because of the underlying efficient data structures, set elements and dictionary keys have to be immutable. Tuples work for this but lists don't.

Several variables can be assigned to the elements of a tuple with one assignment statement. The tuple is said to be *unpacked*. Conversely, if several values (separated by commas) appear on the right side of an assignment statement, they are *packed* into a tuple.

To create a tuple with exactly one element, there must be a comma after that element. Otherwise, something like `(2)` would be ambiguous.

Tuples (Solutions)

1. `t = (1, 2, 3)`
   `t` (1, 2, 3)
2. `()` ()
3. `type(())` <class 'tuple'>
4. `x, y, z = t`
   `x` 1
5. `y` 2
6. `u = x, y`
   `u` (1, 2)
7. `a = 1`
   `b = 2`
   `a, b = b, a`
   `a` 2
8. `b` 1
9. `(2 + 2 + 2)` 6
10. `(2 + 2)` 4
11. `(2)` 2
12. `(2, 2, 2)` (2, 2, 2)
13. `(2, 2)` (2, 2)
14. `(2,)` (2,)